9 Registering services with dependency injection

This chapter covers

- Configuring your services to work with dependency injection
- Choosing the correct lifetime for your services

In chapter 8 you learned about dependency injection (DI) in general, why it's useful as a pattern for developing loosely coupled code, and its central place in ASP.NET Core. In this chapter you'll build on that knowledge to apply DI to your own classes.

You'll start by learning how to configure your app so that the ASP.NET Core framework can create your classes for you, removing the pain of having to create new objects manually in your code. We look at the various patterns you can use to register your services and some of the limitations of the built-in DI container.

Next, you'll learn how to handle multiple implementations of a service. You'll learn how to inject multiple versions of a service, how to override a default service registration, and how to register a service conditionally if you don't know whether it's already registered.

In section 9.4 we look at how you can control for how long your objects are used—that is, their lifetime. We explore the differences among the three lifetime options and some of the pitfalls to be aware of when you come to write your own applications. Finally, in section 9.5 you'll learn why